



Cairo University
Faculty of Computers and Artificial Intelligence

CS213
Object Oriented Programming

Vole Machine Simulator

Object-Oriented Design Evaluation Report

Under the supervision of
Prof. Mohammad El-Ramly

Team Members

Name	ID	Phone Number (Team Leader)
Muhammad Ahmed Noureldin Abdelghany *Team Leader*	20251324	01114102049
Nabil Mohamed Bahget Mohamed	20250699	—
Abdelrahman Haithem Mostafa Said	20250378	—
Eslam Ahmed Abdelaziz Ahmed	20250083	—

October 2026

1. Introduction

This report documents the design phase of the Vole Machine Simulator, the course project of CS213 Object Oriented Programming. Each of the four team members drew a UML class diagram for the same simulator. Before writing any code, the team compared the four designs against a common set of object-oriented design criteria and chose the one that will be implemented.

The report is organised as follows. Section 2 explains the Vole machine: its architecture, specification, instruction set and how a program runs on it. Section 3 states the criteria used to judge a design. Section 4 presents each diagram, explains it class by class, and lists its advantages and disadvantages. Section 5 compares the four diagrams side by side. Section 6 states the chosen design, Abdelrahman's diagram, and the reasons for choosing it.

2. The Vole Machine

2.1 Overview

The Vole machine is a small, hypothetical computer used to teach how a real computer fetches and executes a program. It has a central processing unit (CPU) with a small set of registers, a main memory of 256 one-byte cells, and a simple instruction set of twelve instructions. A program is a list of machine instructions stored in memory; the CPU runs them one after another until it reaches a HALT instruction.

The simulator must reproduce this behaviour in software: load a program from a file into memory, run it step by step or continuously, show the state of memory, registers and screen, and execute every instruction of the instruction set correctly.

2.2 Specification

Component	Specification
CPU	Contains the 16 general-purpose registers, the program counter, the instruction register, the ALU and the control unit.
General-purpose registers	16 registers numbered 0 to F (hexadecimal), each 8 bits wide.
Instruction register (IR)	16 bits. Holds the one instruction currently being executed.
Program counter (PC)	Holds the memory address of the next instruction to fetch. Addresses 00 to FF fit in 8 bits.
Main memory	256 cells of 1 byte (8 bits) each, with addresses 00 to FF.
Instruction size	16 bits, stored in two consecutive memory cells. The first hexadecimal digit is the op-code; the other three digits are the operands. Instructions therefore start at even addresses.
Instruction set	Twelve instructions with op-codes 1 to C (see Section 2.3).
Data representation	Integers use two's complement. Real numbers use an 8-bit floating-point format (1 sign bit, 3-bit exponent, 4-bit mantissa).
Screen	Memory-mapped: storing a value into the cell at address FF writes that value to the screen.

2.3 Instruction set

Every instruction is four hexadecimal digits. In the formats below, R, S and T are register numbers, and XY is an 8-bit value or memory address made of the two digits X and Y.

Op	Format	Meaning	Example
1	RXY	LOAD register R with the contents of the memory cell at address XY.	14A3: R4 ← memory[A3]
2	RXY	LOAD register R with the value XY.	20A3: R0 ← A3
3	RXY	STORE the contents of register R in the memory cell at address XY. If XY is FF, the value is written to the screen.	35B1: memory[B1] ← R5
4	0RS	MOVE the contents of register R to register S.	40A4: R4 ← RA
5	RST	ADD registers S and T as two's complement numbers; put the result in R.	5726: R7 ← R2 + R6
6	RST	ADD registers S and T as floating-point numbers; put the result in R.	634E: R3 ← R4 + RE
7	RST	OR registers S and T; put the result in R.	7CB4: RC ← RB OR R4
8	RST	AND registers S and T; put the result in R.	8045: R0 ← R4 AND R5
9	RST	XOR registers S and T; put the result in R.	95F3: R5 ← RF XOR R3
A	R0X	ROTATE register R one bit to the right, X times. The bit leaving the low end re-enters at the high end.	A403: rotate R4 right 3 times
B	RXY	JUMP to address XY if register R equals register 0; otherwise continue in sequence. The jump is done by copying XY into the PC.	B43C: if R4 = R0 then PC ← 3C
C	000	HALT execution.	C000: stop

2.4 The machine cycle

The CPU repeats a three-phase cycle until a HALT instruction is executed:

- **Fetch.** The control unit reads the two bytes stored at the address in the PC and places them in the instruction register, then adds 2 to the PC so that it points to the next instruction.
- **Decode.** The control unit splits the instruction register into the op-code (first digit) and the operand fields R, S, T, X and Y that the op-code needs.
- **Execute.** The control unit carries out the instruction: it moves data between memory and registers, asks the ALU to compute a result, writes to the screen, changes the PC (JUMP), or stops the machine (HALT).

2.5 Worked example

The following five-instruction program is loaded at address 00. It adds 5 and 3 and shows the result on the screen.

Address	Instruction	Meaning	Effect
00	2005	LOAD R0 with the value 05	R0 = 05
02	2103	LOAD R1 with the value 03	R1 = 03
04	5201	ADD R0 and R1 (two's complement), result into R2	R2 = 08
06	32FF	STORE R2 at address FF	08 is written to the screen
08	C000	HALT	The machine stops

The example uses every part of the machine: memory holds the program, the registers hold the data, the ALU performs the addition, the PC steps through addresses 00, 02, 04, 06, 08, and the screen is reached through the memory-mapped address FF.

3. Evaluation Criteria

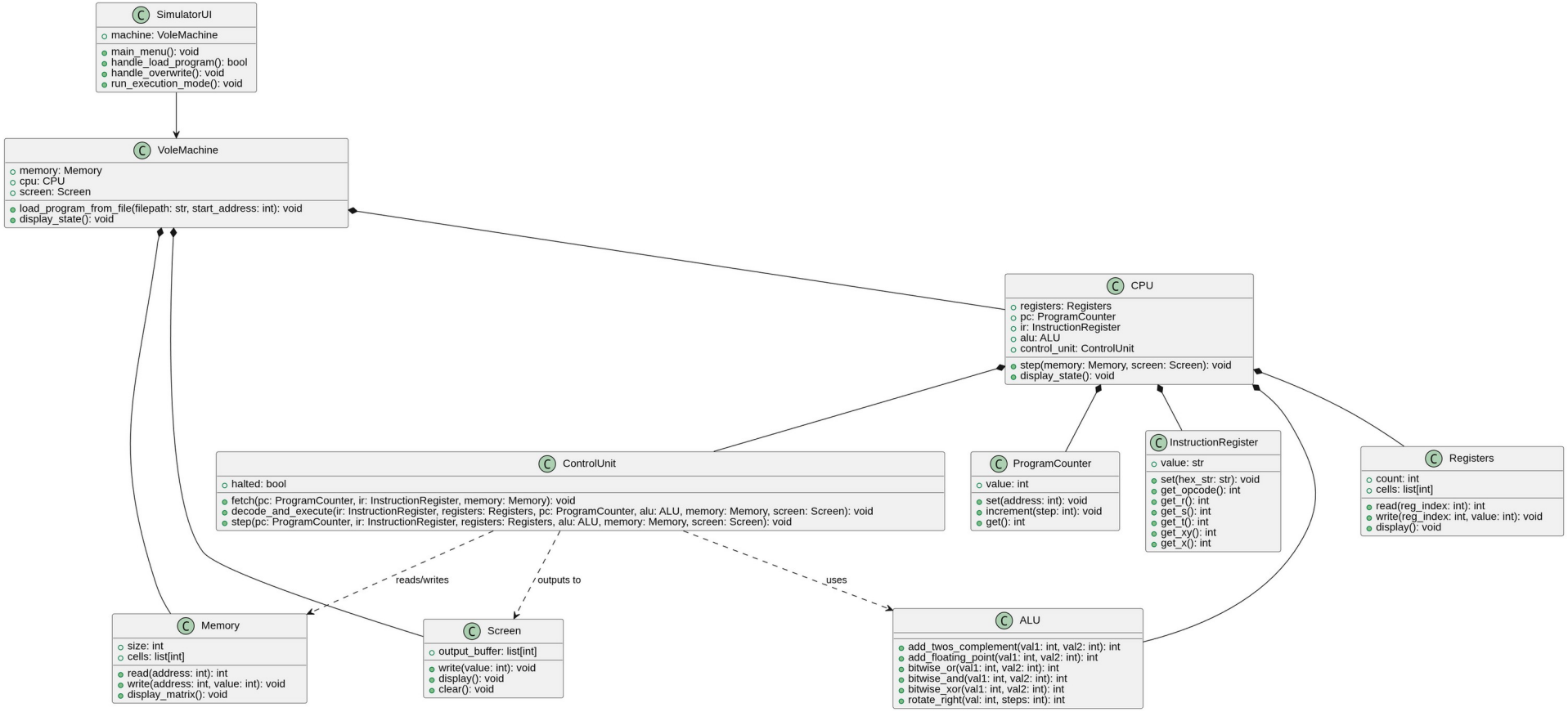
A good object-oriented design for the simulator was defined by the following features. These six criteria are used to judge every diagram in the rest of the report.

- **Completeness.** The classes, attributes, methods and relationships support every function of the simulator.
- **Simplicity.** The design is as simple as possible: not over-engineered, with no unnecessary parts.
- **OOP principles (A-P-I-E).** It applies Abstraction, Polymorphism, Inheritance and Encapsulation where the problem calls for them.
- **Class balance.** Classes are neither too small to justify themselves nor so large that they do several jobs.
- **Domain fidelity.** Classes model the real parts of the Vole machine closely, with additional control and boundary (user-interface) classes.
- **Cohesion and coupling.** Each class has high internal cohesion (one clear purpose) and the coupling between classes is low.

4. Analysis of the Four Diagrams

4.1 Diagram 1: Abdelrahman's design

Figure 1. Class diagram submitted by Abdelrahman Haithem Mostafa Said.



Explanation

This design has ten classes arranged in three levels. SimulatorUI is the boundary class: it shows the menu, loads a program, handles overwriting and runs the execution mode. It holds one VoleMachine. VoleMachine is the top-level object and owns the Memory, the CPU and the Screen. The CPU in turn owns the Registers, the ProgramCounter, the InstructionRegister, the ALU and the ControlUnit, which are exactly the parts of the real machine.

Class	Role and main members
SimulatorUI	Boundary class. Holds the VoleMachine; main_menu(), handle_load_program(), handle_overwrite(), run_execution_mode().
VoleMachine	Whole machine. Owns Memory, CPU and Screen; load_program_from_file(filepath, start_address), display_state().
Memory	256 cells (size, cells); read(address), write(address, value), display_matrix().
CPU	Owns Registers, PC, IR, ALU and ControlUnit; step(memory, screen) runs one cycle; display_state().
Registers	The 16 registers as one array (count, cells); read(reg_index), write(reg_index, value), display().
ProgramCounter	value; set(address), increment(step), get().
InstructionRegister	Holds the instruction as a hexadecimal string; get_opcode(), get_r(), get_s(), get_t(), get_x(), get_xy().
ALU	Six operations: two's complement add, floating-point add, OR, AND, XOR and rotate right.
ControlUnit	Control class. halted flag; fetch(), decode_and_execute(), step().
Screen	output_buffer; write(value), display(), clear().

Relationships: VoleMachine and CPU use composition for the parts they own; ControlUnit has dependency arrows to Memory (reads/writes), ALU (uses) and Screen (outputs to); SimulatorUI has a one-way association to VoleMachine.

Advantages

- Exactly as many classes as the problem needs. Every class is either a real part of the machine (Memory, Registers, ProgramCounter, InstructionRegister, ALU, Screen, CPU, ControlUnit), the whole machine (VoleMachine) or the user interface (SimulatorUI). Nothing is added “just in case”, so it is the simplest of the four designs.
- Closest match to the domain. The CPU, its registers and its units are modelled as the specification describes them, with one control class and one boundary class.
- Balanced classes. No class is trivial and none is overloaded. Modelling the 16 registers as a single Registers class (instead of 16 Register objects) is the simpler and more practical choice.
- High cohesion. The ALU only computes, Memory only stores, Screen only outputs, and the ControlUnit only coordinates the fetch-decode-execute cycle.

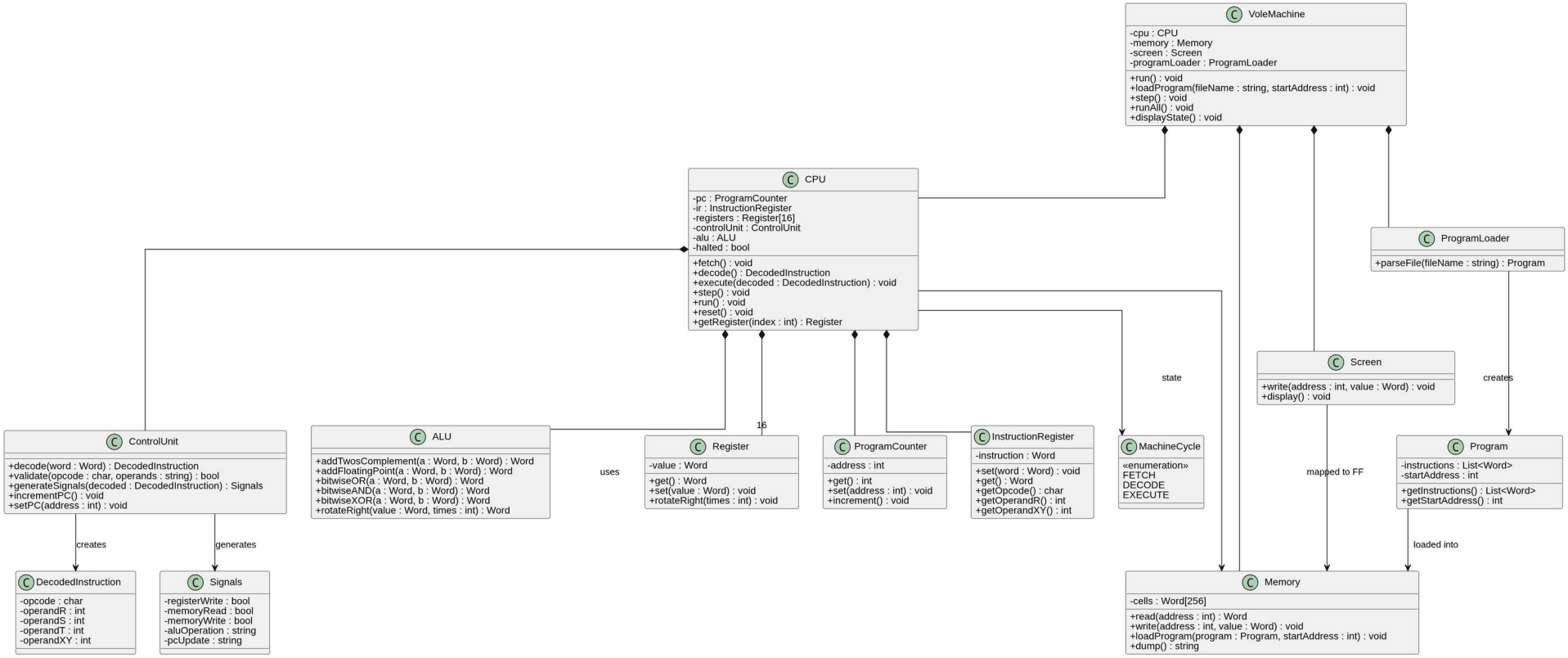
- Low coupling in the diagram. The user interface knows only VoleMachine, and the CPU parts do not know about each other except through the ControlUnit.
- Clear object ownership with composition, and light dependency arrows where a class only uses another.

Disadvantages

- All attributes and methods are public (+). This weakens encapsulation, because any class could change the PC or the memory cells directly.
- decode_and_execute() and step() in ControlUnit take six parameters each. The ControlUnit does not keep references to the components it works with, so every call has to pass them in. This is a coupling smell, and the dependencies on Registers, ProgramCounter and InstructionRegister are not drawn.
- Inheritance and polymorphism are not used, and there is no op-code abstraction such as an enumeration, so the “P” and “I” of A-P-I-E are barely shown. The decode_and_execute() method will contain one large branch over the op-codes.
- Memory and Registers contain display methods (display_matrix(), display()), which mixes presentation with data storage.
- Program loading and file parsing are inside VoleMachine, so validating a program file is not modelled. This keeps the design simple but leaves the checking to the implementation.
- Some attributes are redundant (Registers.count, Memory.size) and the diagram does not state the memory-mapped screen at address FF.

4.2 Diagram 2: Eslam's design

Figure 2. Class diagram submitted by Eslam Ahmed Abdelaziz Ahmed.



Explanation

This design has fourteen classes. VoleMachine is the facade and owns the CPU, Memory, Screen and a ProgramLoader. The CPU owns the ControlUnit, the ALU, the ProgramCounter, the InstructionRegister and sixteen Register objects, and can itself fetch, decode and execute. Around this core the design adds a loading pipeline (ProgramLoader creates a Program) and several objects that describe the decoding process (DecodedInstruction, Signals, MachineCycle). Attributes are private and exposed through methods.

Class	Role and main members
VoleMachine	Facade. Owns CPU, Memory, Screen, ProgramLoader; run(), loadProgram(fileName, startAddress), step(), runAll(), displayState().
CPU	Owns PC, IR, Register[16], ControlUnit, ALU; halted flag; fetch(), decode(), execute(decoded), step(), run(), reset(), getRegister(index).
ControlUnit	decode(word), validate(opcode, operands), generateSignals(decoded), incrementPC(), setPC(address).
ALU	Six operations on Word values (two's complement add, floating-point add, OR, AND, XOR, rotate right).
Memory	Word[256]; read(), write(), loadProgram(program, startAddress), dump().
Register	One register: value; get(), set(), rotateRight(times).
ProgramCounter	address; get(), set(address), increment().
InstructionRegister	instruction; set(word), get(), getOpcode(), getOperandR(), getOperandXY().
Screen	write(address, value), display().
ProgramLoader	parseFile(fileName) returns a Program.
Program	instructions list and startAddress, with two getters.
DecodedInstruction	Holds opcode, operandR, operandS, operandT, operandXY (no methods).
MachineCycle	Enumeration FETCH, DECODE, EXECUTE; associated with the CPU as “state”.
Signals	registerWrite, memoryRead, memoryWrite, aluOperation, pcUpdate; created by ControlUnit.generateSignals().

Relationships: composition from VoleMachine and CPU to the parts they own (CPU *-- 16 Register); CPU uses Memory; ControlUnit creates DecodedInstruction and generates Signals; ProgramLoader creates Program, which is loaded into Memory; Screen is linked to Memory (“mapped to FF”).

Advantages

- Good encapsulation: attributes are private and accessed through methods.
- A clear facade (VoleMachine) with separate run, step and runAll operations, and a ProgramLoader that keeps file handling out of the machine.

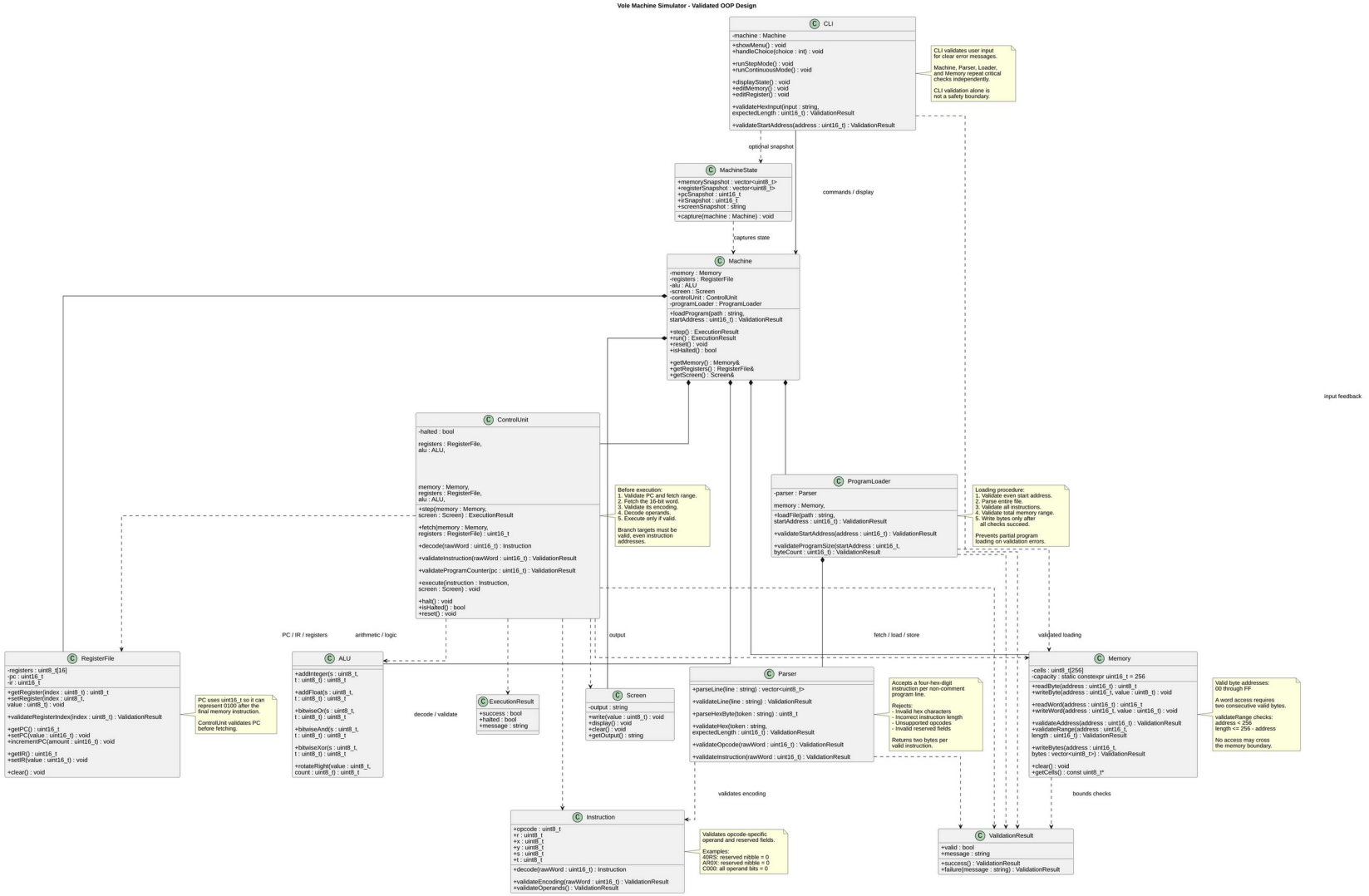
- Each register is an object with its own get, set and rotateRight methods, which is an intuitive mapping of the real hardware.
- The memory-mapped screen at address FF is explicitly mentioned, matching the specification.
- The fetch, decode and execute phases are visible in the design (CPU methods and the MachineCycle enumeration), which helps explain the machine.

Disadvantages

- Over-engineered. Signals, MachineCycle, DecodedInstruction, Program and ProgramLoader are extra classes that the simulator does not need. Nothing in the diagram consumes Signals, and the CPU has no attribute that stores the MachineCycle.
- DecodedInstruction and Signals are anemic: they hold data but have no behaviour (DecodedInstruction has private attributes and no methods at all), so they are classes that are too small.
- Decoding is duplicated. InstructionRegister has operand getters, CPU has decode() and ControlUnit has decode(). The CPU also duplicates fetch, decode and execute that conceptually belong to the ControlUnit, which blurs the split of responsibilities and lowers cohesion.
- ControlUnit exposes incrementPC() and setPC(), but the PC belongs to the CPU and the diagram gives ControlUnit no link to it.
- Screen is linked to Memory, and Program is linked to Memory while Memory.loadProgram() also takes a Program, so Memory and Program depend on each other. The memory-mapping should be a rule applied during STORE, not a link from Screen to Memory. Screen.write() also takes an address, which the screen does not need.
- There is no boundary (user-interface) class, so VoleMachine mixes machine logic with run and display operations.
- The type Word is used throughout but never defined, and ControlUnit.validate() works with a char and a string, which is a weak, string-based design.
- Sixteen Register objects add object overhead for little benefit compared with a single array.

4.3 Diagram 3: Muhammad's design

Figure 3. Class diagram submitted by Muhammad Ahmed Noureldin Abdelghany (team leader).



Explanation

This design, titled “Validated OOP Design”, is built around strict validation. It has thirteen classes. CLI (the boundary) drives a Machine facade, which owns Memory, RegisterFile, ALU, Screen, ControlUnit and ProgramLoader. Two result objects, ValidationResult and ExecutionResult, carry success or failure messages between classes. Program files are read by a ProgramLoader that uses a Parser, and every instruction is checked before it is stored or executed. Notes in the diagram explain each rule.

Class	Role and main members
CLI	Boundary. showMenu(), handleChoice(), runStepMode(), runContinuousMode(), displayState(), editMemory(), editRegister(), plus hex-input and start-address validation.
Machine	Facade. Owns Memory, RegisterFile, ALU, Screen, ControlUnit, ProgramLoader; loadProgram(), step(), run(), reset(), isHalted(), and getters for memory, registers and screen.
Memory	uint8_t[256]; readByte/writeByte, readWord/writeWord, validateAddress, validateRange, writeBytes, clear, getCells.
RegisterFile	16 registers plus the PC and the IR in one class; getters, setters, validateRegisterIndex, incrementPC, clear. PC is 16-bit so it can hold 0100.
ALU	addInteger, addFloat, bitwiseOr, bitwiseAnd, bitwiseXor, rotateRight on 8-bit values.
Instruction	Decoded fields opcode, r, x, y, s, t; decode(rawWord), validateEncoding(), validateOperands() (e.g. reserved digits must be 0).
ControlUnit	halted flag; step(), fetch(), decode(), validateInstruction(), validateProgramCounter(), execute(), halt(), isHalted(), reset().
Screen	output string; write(), display(), clear(), getOutput().
Parser	parseLine(), validateLine(), parseHexByte(), validateHex(), validateOpcode(), validateInstruction().
ProgramLoader	Holds a Parser; loadFile(), validateStartAddress(), validateProgramSize(). Writes bytes only after all checks pass.
ValidationResult	valid, message; success(), failure(message).
ExecutionResult	success, halted, message.
MachineState	Optional snapshot of memory, registers, PC, IR and screen; capture(machine).

Relationships: Machine composes the machine parts; ProgramLoader composes Parser; ControlUnit depends on Memory, RegisterFile, ALU, Screen, Instruction and both result classes; Parser and Memory depend on ValidationResult; CLI depends on Machine, ValidationResult and MachineState.

Advantages

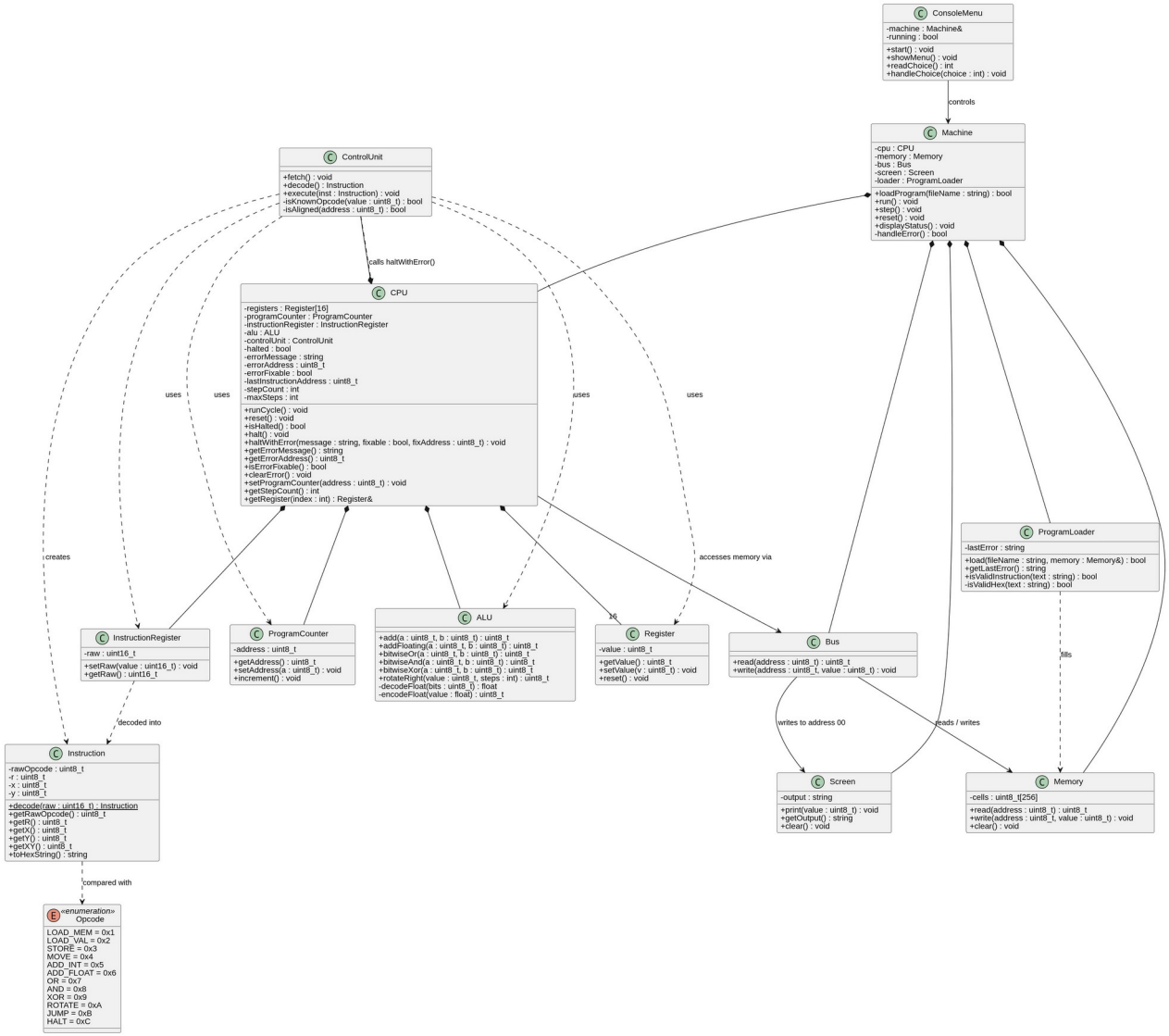
- The most complete and robust design. It handles malformed program files, out-of-range addresses, misaligned jumps, invalid op-codes and reserved fields, and it never loads a program partially.
- Uses exact types (uint8_t for 8-bit cells, uint16_t for the PC) that match the hardware, and thinks about edge cases such as the PC reaching 0100 after the last instruction.
- Clear separation of reading a file (Parser) from placing it in memory (ProgramLoader), and of the user interface (CLI) from the machine.
- The CLI supports step mode, continuous mode, memory editing and register editing, so every function a user could want is covered.
- Self-documenting: the notes record the rules each class enforces.

Disadvantages

- Heavily over-engineered, the clearest violation of the “as simple as possible” criterion. ValidationResult, ExecutionResult, MachineState and Parser are not parts of the Vole machine, and several are too small to justify a class.
- Validation is repeated in many places. Start-address checks exist in both CLI and ProgramLoader; opcode and instruction checks exist in Parser, Instruction and ControlUnit; bounds checks exist in Memory as well. The diagram itself states that critical checks are repeated independently. This duplication is a maintenance risk.
- High coupling: ValidationResult is referenced by almost every class, and ControlUnit depends on five other classes plus two result types.
- Poor domain fidelity. There is no CPU class, and the PC and IR are hidden inside RegisterFile, which therefore mixes three different things (registers, program counter, instruction register) and has low cohesion.
- Fat classes. Memory has nine methods, ControlUnit has nine methods that combine fetching, validating, decoding, executing and halting, and Machine exposes its internals through getMemory(), getRegisters() and getScreen(), which weakens encapsulation.
- The diagram is hard to read: the multi-line method parameters were drawn in the attribute compartments of ControlUnit and ProgramLoader, and many arrows cross.
- Much of the strictness (reserved-nibble checks, aligned branch targets) goes beyond the given specification, so it adds work without being required.

4.4 Diagram 4: Nabil's design

Figure 4. Class diagram submitted by Nabil Mohamed Bahget Mohamed.



Explanation

This design has fourteen elements (thirteen classes and one enumeration). ConsoleMenu is the boundary class and controls the Machine. Machine owns the CPU, Memory, Screen, a Bus and a ProgramLoader. The CPU owns the sixteen Register objects, the ProgramCounter, the InstructionRegister, the ALU and the ControlUnit. The Opcode enumeration names the twelve instructions, and the Bus is placed between the CPU and the memory and screen. The CPU also stores error information so that a faulty program stops with a message.

Class	Role and main members
Opcode	Enumeration of the twelve instructions (LOAD_MEM = 0x1 … HALT = 0xC).
ConsoleMenu	Boundary. Holds a Machine reference; start(), showMenu(), readChoice(), handleChoice().
Machine	Facade. Owns CPU, Memory, Bus, Screen, ProgramLoader; loadProgram(), run(), step(), reset(), displayStatus(), handleError().
CPU	Owns registers, PC, IR, ALU, ControlUnit; halted, error message, error address, fixable flag, step counters; runCycle(), halt(), haltWithError(), error getters, getRegister().
ControlUnit	fetch(), decode(), execute(inst); private isKnownOpcode() and isAligned().
ALU	add, addFloating, bitwiseOr/And/Xor, rotateRight; private decodeFloat() and encodeFloat() helpers.
Register	value; getValue(), setValue(), reset().
ProgramCounter	address; getAddress(), setAddress(), increment().
InstructionRegister	raw 16-bit word; setRaw(), getRaw().
Instruction	Decoded rawOpcode, r, x, y; static decode(raw), getters, toHexString().
Memory	uint8_t[256]; read(), write(), clear().
Bus	read(address), write(address, value); routes accesses to Memory and Screen.
Screen	output string; print(), getOutput(), clear().
ProgramLoader	load(fileName, memory), getLastError(), isValidInstruction(), private isValidHex().

Relationships: Machine and CPU use composition; ControlUnit uses PC, IR, ALU and Register, creates Instruction and calls CPU.haltWithError(); InstructionRegister is decoded into Instruction, which is compared with Opcode; the CPU accesses memory via the Bus; the Bus writes to the Screen and reads/writes Memory; ProgramLoader fills Memory.

Advantages

- Strong encapsulation: attributes are private, helper methods such as isValidHex(), isKnownOpcode(), decodeFloat() and encodeFloat() are private, and access is through public methods.

- The Opcode enumeration gives instructions names instead of magic numbers, a good use of abstraction that makes the execute code readable. It is the only diagram that has one.
- Instruction is a separate decoded value with a static decode() factory, which keeps the InstructionRegister simple (it only stores the raw word).
- The ALU hides the floating-point conversion in private methods, so the complexity of the “6 RST” instruction is contained in one place.
- A clear boundary class (ConsoleMenu) and facade (Machine), and a ProgramLoader that validates files and reports its last error.
- Good user-friendly error handling: the CPU can halt with a message, the address of the fault and whether it can be fixed.

Disadvantages

- The CPU is a fat class: eleven attributes and fourteen methods that mix running the cycle with error reporting (errorMessage, errorAddress, errorFixable, lastInstructionAddress) and run-limiting (stepCount, maxSteps). This lowers cohesion; error handling belongs to a separate concern or to Machine.
- Circular dependency between CPU and ControlUnit: the CPU owns the ControlUnit while the ControlUnit calls back haltWithError() on the CPU. This is tight coupling.
- Bus adds an unnecessary layer. In the real Vole machine the CPU talks to memory directly, and the screen is simply a special memory address, so Bus is an extra class whose two methods only forward calls.
- The diagram says the Bus writes to the screen at address 00, but the specification maps the screen to address FF. This is a mistake that would produce wrong behaviour if implemented as drawn.
- Instruction and InstructionRegister split the same concept in two, and Instruction has no s and t fields. The Opcode link is only “compared with”, so the enumeration is not strongly tied to the design.
- More parts than needed (Bus, Opcode, Instruction, error state, sixteen Register objects). The design is not minimal, though it is less extreme than Diagrams 2 and 3.
- Like the others, it does not use inheritance or polymorphism.

5. Comparison of the Four Diagrams

5.1 Structural comparison

The table below summarises the main structural facts of each diagram. “Helper classes” means classes that do not model a physical part of the machine or the user interface (loader, parser, decoded instruction, bus, enumerations, result and snapshot objects).

Feature	Abdelrahman	Eslam	Muhammad	Nabil
Number of classes	10	14 (incl. 1 enum)	13	14 (incl. 1 enum)
Boundary (UI) class	SimulatorUI	None	CLI	ConsoleMenu
Machine facade	VoleMachine	VoleMachine	Machine	Machine
CPU class	Yes	Yes	No	Yes
Registers	One Registers class	16 Register objects	RegisterFile (also holds PC and IR)	16 Register objects
PC and IR classes	Separate	Separate	Inside RegisterFile	Separate
Attribute visibility	All public	Private	Mostly private	Private
Op-code abstraction	None	None	None	Opcode enumeration
Program loading	Method in VoleMachine	ProgramLoader + Program	ProgramLoader + Parser	ProgramLoader
Validation and errors	Minimal	Limited	Extensive, repeated in many classes	Loader and ControlUnit checks, CPU error state
Helper classes	0	5	6	4
Screen at address FF	Not stated	Stated	Not stated	Stated as 00 (wrong)

5.2 Comparison against the criteria

Each diagram was rated on the six criteria of Section 3 (Excellent = 4, Good = 3, Fair = 2, Weak = 1). The ratings summarise the advantages and disadvantages given in Section 4.

Criterion	Abdelrahman	Eslam	Muhammad	Nabil
Completeness	Good	Good	Excellent	Good
Simplicity	Excellent	Weak	Weak	Fair
OOP principles (A-P-I-E)	Fair	Fair	Fair	Good
Class balance	Good	Fair	Weak	Weak
Domain fidelity	Excellent	Fair	Weak	Good
Cohesion and coupling	Good	Fair	Weak	Fair
Total (out of 24)	**19**	**12**	**10**	**14**

5.3 Discussion

Completeness. All four diagrams cover loading a program, stepping or running it, executing all twelve instructions and displaying the state. Muhammad's design goes furthest by adding validation, memory and register editing and snapshots, but those extras are not required by the simulator's functions.

Simplicity. Abdelrahman's diagram has the fewest classes (10) and no helper classes at all. Eslam's and Muhammad's diagrams add five and six helper classes respectively, and Nabil's adds four. Every extra class must be written, tested and maintained, so the extra parts count against the design when they are not demanded by the specification.

OOP principles. Nabil's design applies abstraction and encapsulation best (Opcode enumeration, private helpers, private attributes). Eslam's and Muhammad's designs use private attributes too, but add abstractions that are not needed. Abdelrahman's design uses public attributes, which is its weakest point. None of the diagrams uses inheritance or polymorphism, and that is acceptable: the Vole machine's parts are different kinds of objects, not variants of a common parent, so forcing a hierarchy (for example one class per instruction) would add classes without making the simulator simpler.

Class balance. Abdelrahman's classes are the most even in size. Nabil's CPU and Muhammad's ControlUnit, Memory and RegisterFile are too large, while DecodedInstruction, Signals, ExecutionResult and ValidationResult are too small.

Domain fidelity. Abdelrahman's diagram is the closest reading of the specification (a CPU with registers, PC, IR, ALU and control unit, plus memory and screen). Muhammad's diagram has no CPU class and merges the PC and IR into the register file, and Nabil's Bus is not a part of the specified machine.

Cohesion and coupling. In Abdelrahman's design each class has one purpose and dependencies are few. Eslam's CPU and ControlUnit overlap, Nabil's CPU and ControlUnit depend on each other in a cycle, and Muhammad's ValidationResult ties almost every class together.

6. Chosen Design and Justification

Decision. The team chose Abdelrahman's diagram (Diagram 1) as the design to implement. It has the highest total in the evaluation (19 out of 24, against 14, 12 and 10), and it is the only design that satisfies the simplicity criterion without losing completeness.

6.1 Reasons

- **It supports every function with the fewest classes.** Loading, stepping, running, displaying state, executing all twelve instructions and reaching the screen are all covered by ten classes. The other designs reach the same functions with 13 or 14 elements.
- **It models the real machine.** Each class matches a part named in the specification: CPU, 16 registers, instruction register, program counter, ALU, control unit, memory and screen. A reader who knows the Vole machine can read the diagram immediately, and the implementation will map directly onto the specification.
- **It has the right extra classes and no more.** SimulatorUI is the boundary class and ControlUnit is the control class, as the design brief requires. There are no result objects, buses, signal objects or snapshot classes.
- **Its classes are balanced and cohesive.** No class mixes unrelated work, and no class is only a data holder.
- **Its weaknesses are easy to correct.** Abdelrahman's disadvantages (public attributes, long parameter lists, missing dependency arrows) can be fixed by editing visibility and constructors, with no new classes. By contrast, the problems of the other designs (the Bus, Signals and the validation layers, the fat CPU, the missing CPU class) are structural and would require redesign.

6.2 Why the other diagrams were not chosen

- **Nabil's design (second place).** It has the best encapsulation and the Opcode enumeration, but its CPU class does too much, the CPU and ControlUnit depend on each other, the Bus is unnecessary, and the screen is mapped to the wrong address (00 instead of FF).
- **Eslam's design (third place).** It is over-engineered with Signals, MachineCycle, DecodedInstruction and Program, duplicates decoding between CPU, ControlUnit and InstructionRegister, ties Screen to Memory, and lacks a boundary class.
- **Muhammad's design (fourth place).** It is the most robust but also the most over-engineered. It has no CPU class, repeats validation across many classes, and carries helper classes the specification does not need.

6.3 Refinements to apply during implementation

The chosen design will be kept as drawn, with the following small changes that borrow the best ideas of the other diagrams without adding classes:

- Make attributes private and access them through the existing getters and setters (from Nabil's and Eslam's designs).
- Let the ControlUnit keep references to the PC, IR, Registers, ALU, Memory and Screen, so that step() and decode_and_execute() need no long parameter lists, and add the missing dependency arrows to the diagram.

- Handle the memory-mapped screen explicitly inside the STORE instruction: a store to address FF calls Screen.write() (as stated in the specification).
- Use named constants or an enumeration for the op-codes 1 to C inside the ControlUnit instead of bare numbers (from Nabil's design).
- Check, inside VoleMachine.load_program_from_file(), that every line is a valid four-digit hexadecimal instruction and that the program fits in memory before copying it (from Muhammad's design), without creating a separate validation class.

7. Conclusion

The four team members produced four valid designs for the Vole Machine Simulator, each with its own strengths: Muhammad's design is the most robust, Nabil's has the best use of encapsulation and abstraction, Eslam's models the fetch-decode-execute process in detail, and Abdelrahman's is the simplest. Judged against the six criteria set for a good object-oriented design, Abdelrahman's diagram gives the best balance of completeness, simplicity, fidelity to the domain and cohesion, and its few weaknesses can be fixed without adding classes. It is therefore the design chosen for implementation, with the refinements listed in Section 6.3.

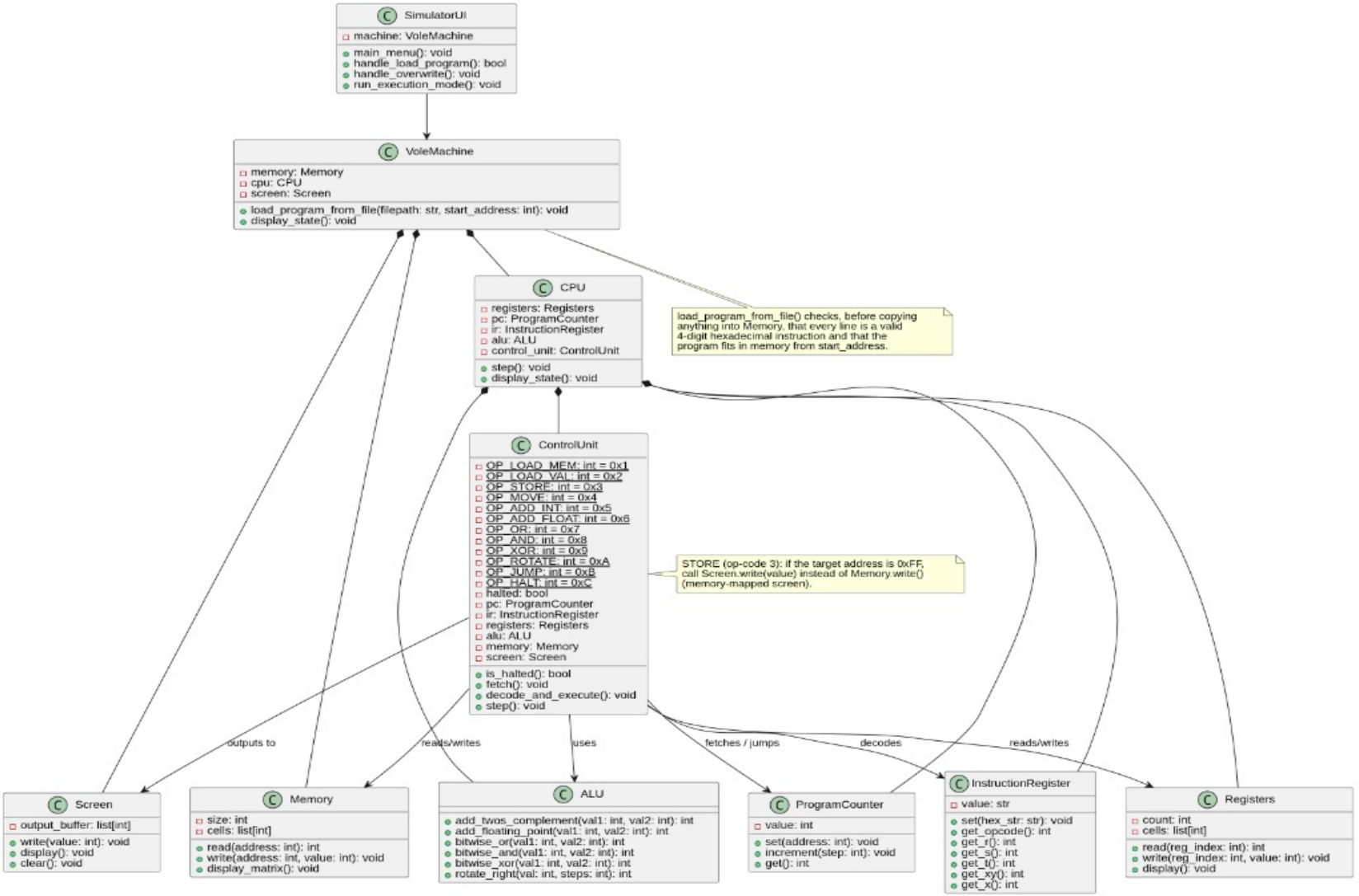
8. Work Breakdown Structure

The table below outlines the responsibilities and contributions of each team member toward the completion of the Vole Machine Simulator project design and report.

Team Member	Role / Responsibilities & Contributions
Muhammad Ahmed Noureldin Abdelghany (Team Leader)	• Created the GitHub repository & project structure • Designed Muhammad's UML class diagram • Authored and structured the comprehensive report • Refined Abdelrahman's diagram to reach the final chosen design
Abdelrahman Haithem Mostafa Said	• Created Abdelrahman's UML class diagram (chosen base design)
Nabil Mohamed Bahget Mohamed	• Created Nabil's UML class diagram • Created explanation video
Eslam Ahmed Abdelaziz Ahmed	• Created Eslam's UML class diagram

9. Final Refined Class Diagram

The diagram below represents the final refined architecture, building upon Abdelrahman's base design with all necessary architectural adjustments and clean integration notes.



10. Required Testing Programs

The following testing programs were implemented and executed to verify the correctness of the Vole Machine Simulator instruction set and execution engine.

Testing Program 1: Check if an immediate value (07) is odd or even using bitwise AND with 01.

```
; Muhammad Ahmed Noureldin Abdelghany ( Team Leader ) 20251324
; Nabil mohamed bahget mohamed 20250699
; Abdelrahman Haithem Mostafa Said 20250378
; Eslam Ahmed Abdelaziz Ahmed 20250083
; ------------------------------------------------------------
; Vole Machine Program
; Purpose: Check if 07 is odd or even using AND with 01.
; Screen shows 01 if odd, 00 if even.
; ------------------------------------------------------------

2107   ; Load immediate value 07 into register 1
2201   ; Load immediate value 01 into register 2
8312   ; AND registers 1 and 2, put result in register 3
33FF   ; Store register 3 to screen (FF)
C000   ; Halt

```

Testing Program 2: Swap registers 1 and 2 (05 and 09) using XOR without a temp register, displaying both.

```
; Muhammad Ahmed Noureldin Abdelghany ( Team Leader ) 20251324
; Nabil mohamed bahget mohamed 20250699
; Abdelrahman Haithem Mostafa Said 20250378
; Eslam Ahmed Abdelaziz Ahmed 20250083
; ------------------------------------------------------------
; Vole Machine Program
; Purpose: Swap registers 1 and 2 (05 and 09) using XOR,
; without a temp register, then display both on the screen.
; ------------------------------------------------------------

2105   ; Load immediate value 05 into register 1
2209   ; Load immediate value 09 into register 2
9112   ; R1 = R1 XOR R2  (05 XOR 09 = 0C)
9221   ; R2 = R2 XOR R1  (09 XOR 0C = 05)
9112   ; R1 = R1 XOR R2  (0C XOR 05 = 09)
31FF   ; Store register 1 to screen (FF): shows 09
32FF   ; Store register 2 to screen (FF): shows 05
C000   ; Halt

```

11. AI Chat Links & Project Video

Below are the clickable references and links to the AI chat sessions utilized during the creation and refinement of the UML diagrams, as well as the project demonstration and explanation video.

Category / Purpose	AI Chat Session / Reference Link
Diagram Creation - Muhammad (ChatGPT)	**Open ChatGPT chat**
Diagram Creation - Eslam (DeepSeek)	**Open DeepSeek chat**
Diagram Creation - Nabil (Claude)	**Open Claude chat**
Diagram Creation - Abdelrahman (Gemini)	**Open Gemini chat**
Diagram Refinement - Muhammad (Claude)	**Open Claude chat**
Project Demonstration & Explanation Video	**Open video link**